STATISTICAL COMPUTING LAB MANUAL

Complete Python Programs, Outputs, Interpretation and Conclusions

Note: The practicals below follow the handwritten Lab Statistics PDF. For hypothesis tests, the assumed reference values are stated explicitly where the sheet does not specify them.

How to run the programs

- Install Python 3 and the required libraries: pip install numpy pandas matplotlib scipy
- Save each practical as a separate .py file, or copy its code into Google Colab/Jupyter Notebook.
- Run one practical at a time. Graph-based practicals open a chart window or display a chart in a notebook.
- The results for random-sampling practicals can vary unless a random seed is set; the supplied code sets a seed for reproducibility.

Practical 1: Create vectors, lists and data frames

Aim

Create vectors, lists and data frames.

Theory

A vector/list stores values in sequence; a DataFrame stores tabular data with named columns.

Python Program

```
import pandas as pd

students = ["Aarav", "Priya", "Rohit", "Sneha", "Kunal", "Divya"]
scores = [78, 85, 92, 67, 74, 88]

# Python list (vector-like structure)
print("Students:", students)
print("Scores:", scores)

# Pandas DataFrame
df = pd.DataFrame({"Student": students, "Score": scores})
print("\nStudent DataFrame:")
print(df)
print("\nData types:")
print(df.dtypes)
```

Expected result / output

Displays the student names and scores as lists and as a two-column DataFrame.

Conclusion

The list and DataFrame representations organize the student-score data clearly.

Practical 2: Calculate mean, median and mode

Aim

Calculate mean, median and mode.

Theory

Mean is the arithmetic average; median is the middle value after sorting; mode is the most frequent value.

Python Program

```
import statistics as st

sales = [42, 38, 51, 29, 46, 34, 61, 40, 37, 44]

print("Sales:", sales)
print("Mean:", st.mean(sales))
print("Median:", st.median(sales))
print("Mode(s):", st.multimode(sales))
```

Expected result / output

Mean = 42.2; median = 41.0. There is no repeated value, so multimode returns every value as a mode under Python's multimode behaviour for all-unique data.

Conclusion

The mean and median summarize the central tendency of the sales observations. Since no value repeats, there is no unique most-frequent observation.

Practical 3: Calculate variance and standard deviation

Aim

Calculate variance and standard deviation.

Theory

Variance measures the spread around the mean. Standard deviation is the square root of variance. Population and sample formulas differ.

Python Program

```
import statistics as st

sales = [42, 38, 51, 29, 46, 34, 61, 40, 37, 44]

print("Mean:", st.mean(sales))
print("Population variance:", st.pvariance(sales))
print("Population standard deviation:", st.pstdev(sales))
print("Sample variance:", st.variance(sales))
print("Sample standard deviation:", st.stdev(sales))
```

Expected result / output

Prints both population and sample variance and standard deviation for the same sales data.

Conclusion

Population measures treat the 10 observations as the whole population; sample measures use the n−1 denominator when the observations represent a sample.

Practical 4: Generate binomial samples and plot the PMF (n=5, p=0.3)

Aim

Generate binomial samples and plot the PMF (n=5, p=0.3).

Theory

A binomial random variable counts successes in a fixed number of independent trials, each with the same success probability.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binom

np.random.seed(42)
n, p = 5, 0.3
samples = np.random.binomial(n=n, p=p, size=1000)
x = np.arange(0, n + 1)
pmf = binom.pmf(x, n, p)

print("First 20 simulated samples:", samples[:20])
print("Sample mean:", round(samples.mean(), 3))
print("Theoretical mean:", n * p)

plt.bar(x, pmf, edgecolor="black")
plt.xticks(x)
plt.xlabel("Number of successes (k)")
plt.ylabel("Probability P(X=k)")
plt.title("Binomial PMF (n=5, p=0.3)")
plt.show()
```

Expected result / output

The bar chart shows P(X=k) for k = 0, 1, 2, 3, 4, 5. The theoretical mean is n×p = 1.5.

Conclusion

The distribution models the number of successes in five trials with success probability 0.3 per trial.

Practical 5: Generate Poisson samples and plot the PMF (lambda=4 complaints/day)

Aim

Generate Poisson samples and plot the PMF (lambda=4 complaints/day).

Theory

The Poisson distribution models the number of events occurring in a fixed interval when the average event rate is λ.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import poisson

np.random.seed(42)
lam = 4
samples = np.random.poisson(lam=lam, size=1000)
x = np.arange(0, 13)
pmf = poisson.pmf(x, mu=lam)

print("First 20 simulated daily counts:", samples[:20])
print("Simulated mean:", round(samples.mean(), 3))
print("Theoretical mean:", lam)

plt.bar(x, pmf, edgecolor="black")
plt.xticks(x)
plt.xlabel("Complaints per day")
plt.ylabel("Probability")
plt.title("Poisson PMF (lambda=4)")
plt.show()
```

Expected result / output

The chart shows probabilities for 0 to 12 complaints. The theoretical mean number of complaints per day is 4.

Conclusion

The Poisson model represents daily complaint counts when the average rate is four complaints per day.

Practical 6: Define and visualize a non-normal population

Aim

Define and visualize a non-normal population.

Theory

The supplied values are 10, 20, 30, 40 and 200. The value 200 creates a strong right tail, so this small illustrative population is highly right-skewed rather than normally distributed.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt

population = np.array([10, 20, 30, 40, 200])

print("Population:", population)
print("Mean:", population.mean())
print("Median:", np.median(population))

plt.hist(population, bins=[0, 25, 50, 75, 100, 125, 150, 175, 200, 225],
         edgecolor="black")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Illustrative right-skewed, non-normal population")
plt.show()
```

Expected result / output

The mean is 60 and the median is 30. The large value 200 pulls the mean above the median.

Conclusion

The population is not symmetric; the extreme high value produces right-skew. With only five values, this is an illustration rather than a formal distribution test.

Practical 7: Compute 90%, 95% and 99% confidence intervals for the mean

Aim

Compute 90%, 95% and 99% confidence intervals for the mean.

Theory

Because population standard deviation is not given and the sample is small, use a two-sided Student t confidence interval: mean ± t-critical × sample standard error.

Python Program

```
import numpy as np
from scipy import stats

data = np.array([248, 251, 247, 253, 250, 249, 252, 246, 254, 250])
n = len(data)
mean = data.mean()
sd = data.std(ddof=1)
se = sd / np.sqrt(n)

print("n =", n)
print("Mean =", mean)
print("Sample standard deviation =", sd)
for confidence in [0.90, 0.95, 0.99]:
    alpha = 1 - confidence
    critical_t = stats.t.ppf(1 - alpha/2, df=n-1)
    low = mean - critical_t * se
    high = mean + critical_t * se
    print(f"{confidence:.0%} CI: ({low:.3f}, {high:.3f})")
```

Expected result / output

Prints the sample mean and the 90%, 95% and 99% confidence intervals.

Conclusion

Higher confidence levels produce wider intervals because more confidence requires a broader range of plausible population means.

Practical 8: Perform a one-sample t-test

Aim

Perform a one-sample t-test.

Theory

The sheet gives measurements 498, 502, 497, 501, 499, 503, 496 and 500 but does not specify a hypothesized mean. This program explicitly tests H0: population mean = 500 against a two-sided alternative.

Python Program

```
import numpy as np
from scipy import stats

data = np.array([498, 502, 497, 501, 499, 503, 496, 500])
hypothesized_mean = 500

t_stat, p_value = stats.ttest_1samp(data, popmean=hypothesized_mean)
print("Sample mean:", data.mean())
print("Hypothesized mean:", hypothesized_mean)
print("t-statistic:", round(t_stat, 4))
print("p-value:", round(p_value, 4))

alpha = 0.05
if p_value < alpha:
    print("Decision: Reject H0 at the 5% significance level.")
else:
    print("Decision: Fail to reject H0 at the 5% significance level.")
```

Expected result / output

Tests whether the sample provides evidence that its population mean differs from 500 at the 5% significance level.

Conclusion

If p-value is below 0.05, reject H0; otherwise, there is insufficient evidence to conclude that the mean differs from 500. This is not proof that the mean equals 500.

Practical 9: Perform an independent two-sample t-test

Aim

Perform an independent two-sample t-test.

Theory

Compare the mean scores of Shift A and Shift B. Welch's t-test is used because it does not require equal population variances.

Python Program

```
import numpy as np
from scipy import stats

shift_a = np.array([45, 48, 42, 50])
shift_b = np.array([52, 55, 50, 58])

print("Shift A mean:", shift_a.mean())
print("Shift B mean:", shift_b.mean())

t_stat, p_value = stats.ttest_ind(shift_a, shift_b, equal_var=False)
print("Welch t-statistic:", round(t_stat, 4))
print("p-value:", round(p_value, 4))

if p_value < 0.05:
    print("Decision: Significant difference at 5%.")
else:
    print("Decision: No statistically significant difference at 5%.")
```

Expected result / output

Displays each shift's mean, Welch t-statistic and p-value.

Conclusion

The test checks whether the population means differ. Interpret the p-value using a 5% significance level.

Practical 10: Compute regression coefficients using least squares and verify with Python

Aim

Compute regression coefficients using least squares and verify with Python.

Theory

Fit the straight-line model y = a + bx, where x is TV advertising spend and y is sales. Least squares chooses a and b to minimize the sum of squared residuals.

Python Program

```
import numpy as np
from scipy.stats import linregress
import matplotlib.pyplot as plt

x = np.array([100, 150, 200, 250, 300], dtype=float)  # TV spend
y = np.array([150, 200, 230, 280, 320], dtype=float)  # Sales

x_mean = x.mean()
y_mean = y.mean()

# Least-squares formulas
slope_formula = np.sum((x - x_mean) * (y - y_mean)) / np.sum((x - x_mean)**2)
intercept_formula = y_mean - slope_formula * x_mean

# Verify with scipy
result = linregress(x, y)

print("Slope (formula):", round(slope_formula, 4))
print("Intercept (formula):", round(intercept_formula, 4))
print("Slope (SciPy):", round(result.slope, 4))
print("Intercept (SciPy):", round(result.intercept, 4))
print("Correlation r:", round(result.rvalue, 4))
print("R-squared:", round(result.rvalue**2, 4))
print("Predicted sales at TV spend 220:",
      round(intercept_formula + slope_formula * 220, 2))

plt.scatter(x, y, label="Observed data")
plt.plot(x, intercept_formula + slope_formula*x, label="Least-squares line")
plt.xlabel("TV spend")
plt.ylabel("Sales")
plt.title("TV Spend vs Sales: Linear Regression")
plt.legend()
plt.show()
```

Expected result / output

Prints slope, intercept, correlation, R-squared and predicted sales at TV spend 220; plots observations and fitted line.

Conclusion

A positive slope indicates sales increase as TV spend increases in this dataset. R-squared describes the proportion of observed sales variation explained by the fitted linear model; it does not establish causation.

Important notes

Practical 2: because every sales value is distinct, there is no unique mode. Python's statistics.multimode returns all values when every value occurs equally often.

Practical 6: the five values are intentionally treated as a small illustrative right-skewed population, not as a large empirical sample.

Practical 8: the hypothesized mean of 500 is an explicit assumption because the lab sheet does not state a null mean.

Practical 10: the regression input data are read from the second page of the supplied lab sheet: TV spend = 100, 150, 200, 250, 300 and sales = 150, 200, 230, 280, 320.